

Documento 5

Chat Development Guide

Sistema Integral de Gestión Comercial para una Importadora Mayorista

Manual oficial para desarrollar el proyecto usando ChatGPT Projects, Flask, Git y Pull Requests

Campo	Descripción
Versión	1.0 - Guía metodológica oficial
Fecha	Julio 2026
Estado	Fuente oficial de metodología de trabajo
Uso	Guía para Project Manager, documentación, desarrollo, revisión, Git y chats por módulo
Alcance	Uso de ChatGPT Projects, organización de chats, flujo de sprints, Git Workflow, Pull Requests, checklists y prompts maestros
Fuentes base	Documento 0, Documento 2, Documento 3, Documento 4 y Documento 6

Restricción principal

Este documento no modifica reglas de negocio, arquitectura, tecnologías, estructura modular, flujo Git ni decisiones aprobadas. Su función es definir la forma oficial de trabajar durante todo el desarrollo.

Índice

Sección	Título
0	Fuentes oficiales y criterios de interpretación
1	Objetivo del documento
2	¿Por qué utilizar un Proyecto de ChatGPT?
3	Organización oficial del Proyecto
4	Función del Project Manager
5	Cómo debe utilizarse el Project Manager
6	Cuándo abrir un nuevo chat
7	Convención oficial para nombrar chats
8	Flujo oficial de trabajo
9	Cómo iniciar un chat de desarrollo
10	Cómo cerrar un chat
11	Qué debe entregar un chat antes de cerrarse
12	Cómo actualizar el Project Manager
13	Cómo revisar código
14	Cómo solicitar cambios
15	Cómo mantener la coherencia entre módulos
16	Git Workflow
17	Convenciones para Branches
18	Convenciones para Commits
19	Convenciones para Pull Requests
20	Buenas prácticas
21	Errores que deben evitarse
22	Checklist antes de iniciar un Sprint
23	Checklist antes de cerrar un Sprint
24	Recomendaciones para trabajar con ChatGPT
25	Biblioteca Oficial de Prompts
26	Cierre operativo

Nota

Este índice es estructural. La paginación final queda determinada por la exportación DOCX/PDF.

0. Fuentes oficiales y criterios de interpretación

Este documento se construye como la metodología oficial de trabajo para el proyecto. Debe leerse junto con los documentos base cargados en el Proyecto de ChatGPT. Cuando exista duda, prevalecen el ADR y el Roadmap para decisiones y ejecución.

Fuente oficial	Uso en esta guía
Documento 0 - Project Context	Define propósito, alcance, módulos principales, reglas de negocio, estados y forma de trabajar por chats.
Documento 2 - Arquitectura Técnica	Define Flask, Jinja2, Bootstrap, CSS propio, SQLite, Blueprints, estructura modular, capas y Git Flow.
Documento 3 - Design System	Define identidad visual, componentes, tokens, responsive design, accesibilidad y consistencia UI/UX.
Documento 4 - Development Roadmap	Define fases, sprints, branches, Definition of Done, orden de construcción y plan de chats.
Documento 6 - ADR	Registra decisiones aprobadas y regla que ningún chat de desarrollo modifica decisiones sin pasar primero por el chat MASTER.

Criterio de coherencia

- Si una indicación contradice el ADR, se detiene el trabajo y se consulta al Project Manager.
- Si un cambio impacta código, UI, base de datos, roadmap o SDD, primero debe registrarse en el Project Manager y, si corresponde, en el ADR.
- Los chats de desarrollo no redefinen reglas de negocio ni tecnologías; solo implementan el alcance aprobado.
- El producto sigue siendo la fuente única para catálogo, inventario, cotizaciones y ventas.
- El inventario solo se descuenta cuando un pedido queda pagado; una cotización no descuenta inventario.

1. Objetivo del documento

El objetivo de este documento es explicar cómo desarrollar el Sistema Integral de Gestión Comercial utilizando un Proyecto de ChatGPT sin perder contexto entre conversaciones. La guía define roles, chats, flujo de trabajo, Git Workflow, checklists y cuatro prompts maestros reutilizables.

La meta operativa es simple: cada conversación nueva debe poder continuar el proyecto usando las fuentes oficiales, el Roadmap, el ADR y esta guía, sin inventar decisiones ni mezclar módulos.

Principio rector

Un sprint = un objetivo claro = un chat de trabajo = una branch feature = un Pull Request = una actualización al Project Manager.

Enfoque	Resultado esperado
Para el equipo	Alinear la forma de trabajar, reducir retrabajo y facilitar incorporación de nuevos desarrolladores.
Para ChatGPT	Establecer instrucciones repetibles para documentación, desarrollo, Project Manager y revisión.
Para Git	Mantener ramas, commits y Pull Requests ordenados según

Para la documentación	Garantizar que decisiones, avances y cambios queden trazables en el Project Manager y documentos oficiales.
------------------------------	---

2. ¿Por qué utilizar un Proyecto de ChatGPT?

El sistema se desarrolla por módulos y requiere mantener consistencia entre reglas de negocio, arquitectura técnica, UI/UX, roadmap y decisiones aprobadas. Un Proyecto de ChatGPT permite cargar esos documentos como fuentes oficiales y abrir conversaciones independientes sin repetir todo el contexto desde cero.

Beneficios prácticos

Beneficio	Aplicación en el proyecto
Contexto permanente	Cada chat puede consultar documentos base: Project Context, Arquitectura Técnica, Design System, Roadmap y ADR.
Trabajo modular	Un chat puede concentrarse en Productos, Inventario, Ventas, Cotizaciones, Dashboard o Deploy.
Menos contradicciones	Las decisiones aprobadas se mantienen en el ADR y el Project Manager.
Mejor trazabilidad	Cada sprint genera resumen, branch, commit, PR y actualización de estado.
Incorporación rápida	Un desarrollador nuevo puede revisar documentos oficiales y usar el prompt maestro correspondiente.

Qué no reemplaza

- No reemplaza Git, GitHub ni los Pull Requests.
- No reemplaza las pruebas locales del sistema Flask.
- No reemplaza el criterio del equipo para validar reglas de negocio.
- No autoriza cambios de arquitectura sin aprobación en el Project Manager y ADR.

3. Organización oficial del Proyecto

El Proyecto de ChatGPT se organiza con una fuente central de documentos, un chat Project Manager que actúa como chat MASTER y chats especializados para documentación, desarrollo y revisión.

Mapa oficial del Proyecto en ChatGPT

Fuentes oficiales, chat Project Manager y chats de trabajo



Figura 1. Organización oficial del Proyecto de ChatGPT.

Elemento	Responsabilidad
Fuentes del Proyecto	Documentos oficiales cargados en ChatGPT Projects. Son el contexto base de todos los chats.
★ PM - Project Manager	Chat MASTER. Controla estado, decisiones, Roadmap, ADR, SDD y próximo sprint.
DOC	Chats para crear o actualizar documentos oficiales usando Prompt A.
DEV	Chats para construir un módulo o pantalla usando Prompt B.
REV	Chats para revisar código, arquitectura o Pull Requests usando Prompt D.
GitHub	Repositorio con ramas main, dev y feature/<módulo>, commits en inglés y Pull Requests.

Regla de organización

Los chats no se organizan por ocurrencias improvisadas. Se organizan por objetivo: documentación, desarrollo, Project Manager o revisión.

4. Función del Project Manager

El Project Manager es el chat MASTER del proyecto. Su función es mantener el estado general, registrar cambios relevantes, decidir el próximo sprint y garantizar que los chats de desarrollo no pierdan coherencia.

Función	Qué controla
Estado general	Fase actual, sprint actual, módulos terminados, módulos pendientes y bloqueos.

Roadmap	Avance por fases y sprints, Definition of Done y próximo trabajo.
ADR	Decisiones nuevas o cambios aprobados con fecha, justificación e impacto.
SDD	Actualizaciones funcionales cuando una decisión afecte reglas, flujos o alcance.
Riesgos	Contradicciones entre chats, deuda técnica, módulos incompletos o cambios no aprobados.
Próximo Sprint	Qué chat abrir, qué prompt usar, qué branch crear y cuál es la salida esperada.

Responsabilidad del Project Manager

- Mantener una visión única del proyecto completo.
- Recibir resúmenes de cierre de cada chat de desarrollo.
- Detectar si un cambio requiere actualizar ADR, SDD, Roadmap o documentación técnica.
- Evitar que se abran chats redundantes o con objetivos demasiado amplios.
- Indicar el siguiente sprint cuando un sprint anterior queda cerrado.

5. Cómo debe utilizarse el Project Manager

El Project Manager se utiliza al inicio y al cierre de cada sprint, y cada vez que exista una decisión que pueda impactar el proyecto completo. No debe usarse para programar un módulo completo; para eso existe el Prompt B de desarrollo.

Momento	Acción en el PM	Salida esperada
Inicio del proyecto	Confirmar documentos fuente, fase actual y primer sprint.	Plan de trabajo inmediato.
Antes de iniciar un sprint	Revisar Roadmap, dependencias y DoD.	Chat, branch y alcance aprobados.
Después de cerrar un chat DEV	Registrar resumen, archivos modificados, PR, pruebas y pendientes.	Roadmap y estado actualizados.
Si aparece una decisión	Evaluar impacto y registrar en ADR si corresponde.	Decisión trazable antes de implementar.
Antes del siguiente sprint	Determinar si se continúa, se corrige o se integra.	Siguiente paso claro.

Uso correcto

El PM no resuelve todo dentro de un solo chat. Su trabajo es coordinar, no mezclar desarrollo, revisión y documentación en una misma conversación.

6. Cuándo abrir un nuevo chat

Un nuevo chat se abre cuando cambia el objetivo de trabajo. La regla es evitar chats gigantes que mezclen productos, inventario, ventas, UI, deploy y revisión al mismo tiempo.

Situación	¿Nuevo chat?	Qué prompt usar
Crear o actualizar un documento oficial	Sí	Prompt A - Documentación

Iniciar un módulo o pantalla nueva	Sí	Prompt B - Desarrollo
Continuar el mismo módulo sin cerrar PR	No	Seguir en el chat DEV actual
Actualizar estado general después de un cierre	No abrir DEV; usar PM	Prompt C - Project Manager
Revisar un Pull Request o código terminado	Sí, si la revisión es significativa	Prompt D - Revisión
Corregir cambios solicitados del mismo PR	Preferible mismo chat DEV o REV	Prompt B o D según el caso
Cambiar una decisión aprobada	Usar PM antes de desarrollar	Prompt C - Project Manager

Criterio simple

Regla

Si el chat necesita otro objetivo, otro prompt o otra branch, debe ser otro chat.

7. Convención oficial para nombrar chats

Los nombres de chats deben permitir identificar rápidamente el tipo de trabajo, el objetivo y el estado del proyecto. Se mantiene una convención breve, visual y constante.

Tipo	Nombre oficial	Uso
Project Manager	★ PM - Project Manager	Estado general, decisiones, Roadmap, ADR, SDD y próximo sprint.
Documentación	📄 DOC - Arquitectura Técnica	Documento técnico oficial.
Documentación	🎨 DOC - Design System	Guía UI/UX oficial.
Documentación	📖 DOC - Chat Development Guide	Este documento metodológico.
Desarrollo	💻 DEV - Configuración Flask	Repositorio, entorno, Flask y estructura modular.
Desarrollo	💻 DEV - Productos	CRUD de productos, estados, imágenes y validaciones.
Desarrollo	💻 DEV - Inventario	Stock, movimientos, ajustes y bajo stock.
Desarrollo	💻 DEV - Ventas	Pedidos, pago confirmado, comprobantes e inventario.
Desarrollo	💻 DEV - Cotizaciones	Cotizaciones, resumen y posible conversión a pedido.
Desarrollo	💻 DEV - Dashboard	Indicadores administrativos y actividad general.
Desarrollo	💻 DEV - Deploy	Publicación, README y ajustes finales.
Revisión	🔍 REV - PR Productos	Revisión de código, arquitectura, reglas y Pull Request.

Reglas de nombres

- El prefijo indica el tipo de trabajo: PM, DOC, DEV o REV.
- El nombre debe ser corto y relacionado con el Roadmap.
- No usar nombres ambiguos como 'ayuda', 'continuación' o 'varios cambios'.

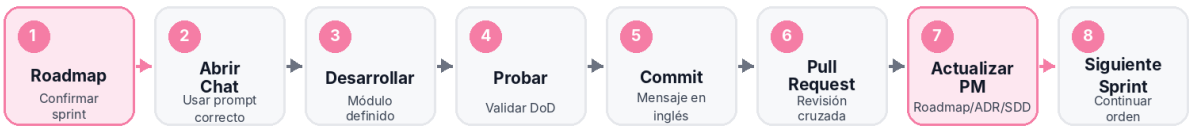
- No abrir un chat DEV con más de un módulo principal si no es una fase de integración aprobada.

8. Flujo oficial de trabajo

El flujo oficial transforma el Roadmap en trabajo ejecutable. Todo módulo debe avanzar por el mismo circuito para conservar trazabilidad: Roadmap, chat, desarrollo, prueba, commit, Pull Request, actualización del PM y siguiente sprint.

Flujo oficial de trabajo

Roadmap → chat → desarrollo → revisión → actualización



Cada sprint debe terminar con código probado, commit, Pull Request y resumen para el Project Manager.

Figura 2. Flujo oficial de trabajo del proyecto.

Paso	Actividad	Criterio
1	Roadmap	Revisar fase, sprint, DoD, dependencias y branch sugerida.
2	Abrir Chat	Crear chat con nombre oficial y usar el prompt maestro correcto.
3	Desarrollar	Implementar solo el alcance del módulo o pantalla.
4	Probar	Validar reglas, UI, responsividad, errores y dependencias.
5	Commit	Registrar cambios con mensajes descriptivos en inglés.
6	Pull Request	Abrir PR hacia dev y solicitar revisión cruzada.
7	Actualizar Project Manager	Registrar avance, decisiones, PR, pruebas y pendientes.
8	Siguiente Sprint	El PM confirma si se continúa, corrige, integra o despliega.

Regla diaria

feature/* → Pull Request → dev. Nunca feature/* → main.

9. Cómo iniciar un chat de desarrollo

Un chat de desarrollo inicia con el Prompt B - Desarrollo. El usuario solo cambia el nombre del módulo, sprint o pantalla. Antes de pedir código, el chat debe confirmar alcance, branch y Definition of Done.

Pasos de apertura

1. Abrir un nuevo chat con nombre oficial, por ejemplo:  DEV - Productos.
2. Pegar el Prompt B - Desarrollo y reemplazar [NOMBRE_DEL_MÓDULO], [SPRINT] y [BRANCH].
3. Solicitar primero un plan breve de implementación alineado al Roadmap.
4. Confirmar qué archivos tocará el módulo y qué dependencias necesita.
5. Desarrollar en pasos pequeños, revisando código y pruebas después de cada parte importante.
6. Mantener el alcance del chat; si surge una decisión mayor, detener y pasar al Project Manager.

Datos mínimos de entrada

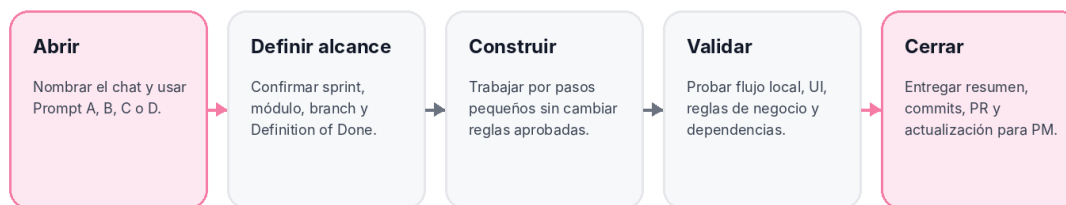
Dato	Ejemplo
Módulo	Productos
Sprint	4.3 Productos
Branch	feature/admin-products
DoD	Alta, edición, archivo e imágenes funcionando.
Documentos a respetar	Documento 0, 2, 3, 4 y 6.

10. Cómo cerrar un chat

Cerrar un chat significa dejar una transferencia clara hacia GitHub y el Project Manager. No basta con decir que el código está listo; el chat debe entregar evidencia mínima de avance.

Ciclo de vida de un chat

Apertura, ejecución, cierre y transferencia al Project Manager



Un chat no se considera cerrado hasta que su resultado se pueda copiar al Project Manager sin reconstruir contexto.

Figura 3. Ciclo de vida de un chat de trabajo.

Pasos de cierre

7. Validar que el módulo cumple su Definition of Done.
8. Probar localmente el flujo afectado y registrar qué se probó.
9. Preparar commits descriptivos en inglés.
10. Abrir Pull Request hacia dev.
11. Solicitar revisión cruzada o revisión con Prompt D.
12. Generar resumen de cierre para copiar al Project Manager.
13. Actualizar el Project Manager con Prompt C antes de iniciar el siguiente sprint.

Salida obligatoria

Todo cierre debe dejar un resumen que permita continuar el proyecto sin leer todo el historial del chat.

11. Qué debe entregar un chat antes de cerrarse

Cada chat de desarrollo debe entregar un paquete mínimo de cierre. Este paquete no reemplaza el Pull Request, pero facilita la trazabilidad en el Project Manager.

Entregable	Contenido mínimo
Resumen funcional	Qué se construyó y qué problema resuelve.
Archivos modificados	Lista de rutas, templates, modelos, servicios, CSS o configuración tocados.
Reglas respetadas	Decisiones del ADR y reglas de negocio aplicadas.
Pruebas realizadas	Flujos probados localmente, validaciones y casos negativos importantes.
Branch	Nombre exacto de la branch feature.
Commits	Mensajes finales en inglés.
Pull Request	Título, descripción, checklist y destino dev.
Pendientes	Bloqueos, mejoras o decisiones que deben pasar al Project Manager.
Siguiente sugerido	Próximo sprint o revisión necesaria según Roadmap.

Formato de resumen de cierre

Resumen de cierre - [Módulo]

Sprint: [número y nombre]

Branch: feature/[nombre]

Pull Request: [título o enlace]

1. Qué se construyó:

- ...

2. Archivos modificados:

- ...

3. Reglas del proyecto respetadas:

- ...

4. Pruebas realizadas:

- ...

5. Pendientes o riesgos:

- ...

6. Recomendación para Project Manager:

- Marcar sprint como [completo / en revisión / bloqueado].
- Próximo sprint sugerido: [...]

12. Cómo actualizar el Project Manager

El Project Manager se actualiza con Prompt C. La actualización debe ser breve, verificable y útil para decidir el siguiente paso. No se copian todos los mensajes del chat; se copia el resumen de cierre.

Contenido de una actualización

- ☐ Fase y sprint actual.
- ☐ Nombre del chat DEV o REV cerrado.
- ☐ Branch y Pull Request asociados.
- ☐ Estado del DoD: completo, parcial o bloqueado.
- ☐ Pruebas realizadas y resultado.
- ☐ Decisiones nuevas o cambios que podrían impactar ADR, SDD, Roadmap o Arquitectura Técnica.
- ☐ Pendientes y riesgos.
- ☐ Próximo sprint sugerido.

Plantilla de actualización al PM

Actualización para Project Manager

Fase: [Fase del Roadmap]

Sprint: [Sprint]

Chat cerrado: [Nombre oficial del chat]

Branch: [feature/...]

Pull Request: [título o enlace]

Estado: [Completo / En revisión / Bloqueado]

Resumen:

- ...

Pruebas:

- ...

Decisiones o impacto documental:

- ADR: [sin cambios / requiere actualización]
- Roadmap: [sin cambios / actualizar estado]
- SDD: [sin cambios / requiere actualización]
- Arquitectura o Design System: [sin cambios / requiere revisión]

Pendientes:

- ...

Próximo sprint sugerido:

- ...

13. Cómo revisar código

La revisión de código se realiza con Prompt D - Revisión. Debe validar que el cambio cumple el Roadmap, respeta la arquitectura modular, no rompe reglas de negocio y mantiene coherencia visual con el Design System.

Área de revisión	Preguntas clave
Arquitectura	¿Las rutas, servicios, modelos, templates y static están separados? ¿Se respeta Flask modular con Blueprints?
Negocio	¿Producto sigue siendo fuente única? ¿Inventario solo descuenta al pagar? ¿No se elimina físicamente producto?
UI/UX	¿Usa Bootstrap y CSS propio? ¿Respeto tokens, badges, tablas, cards y responsive design?
Seguridad	¿Las vistas internas están protegidas por rol? ¿El público no ve datos sensibles ni stock exacto?
Datos	¿No duplica producto? ¿Registra movimientos con usuario, fecha y motivo cuando aplica?
Calidad	¿El código es legible, mantenible y no concentra lógica de negocio en templates?
Git	¿La branch sale de dev? ¿El PR apunta a dev? ¿Los commits están en inglés?

Niveles de hallazgo

Nivel	Cuándo usar	Acción
Bloqueante	Rompe una regla aprobada, flujo principal, seguridad, inventario o Git Flow.	No aprobar PR.
Importante	Funciona, pero genera deuda técnica, inconsistencia UI o validación incompleta.	Solicitar cambios antes de merge.
Mejora	Sugerencia de claridad, estilo o mantenibilidad sin bloquear el flujo.	Puede aprobarse con recomendación.

14. Cómo solicitar cambios

Los cambios solicitados deben ser específicos, accionables y verificables. Una revisión no debe limitarse a decir 'mejorar código'; debe indicar qué archivo, qué problema y cuál es el criterio de aceptación.

Campo	Descripción
Archivo o zona	Ruta, función, template, servicio o pantalla donde aparece el problema.
Problema	Qué está mal y qué regla del proyecto afecta.

Cambio requerido	Qué debe modificarse para cumplir la guía.
Criterio de aceptación	Cómo se validará que el cambio quedó correcto.
Prioridad	Bloqueante, Importante o Mejora.

Plantilla de solicitud de cambios

Solicitud de cambio

Prioridad: [Bloqueante / Importante / Mejora]

Archivo o zona: [ruta o pantalla]

Problema:

- ...

Regla afectada:

- [ADR / Roadmap / Arquitectura / Design System / regla de negocio]

Cambio requerido:

- ...

Criterio de aceptación:

- ...

Regla de equipo

Una solicitud de cambio debe ayudar a corregir, no solo señalar. Siempre debe incluir el resultado esperado.

15. Cómo mantener la coherencia entre módulos

La coherencia se mantiene respetando las mismas reglas, nombres, modelos, servicios y componentes visuales en todos los módulos. El sistema debe sentirse como un producto único, no como pantallas aisladas.

Regla de coherencia	Aplicación práctica
Producto único	Catálogo, inventario, cotizaciones y ventas consultan el mismo Product; no se crean copias.
Inventario	Solo ventas pagadas descuentan stock; cotizaciones y pedidos pendientes no descuentan.
Catálogo público	Muestra disponibilidad pública, no cantidades exactas.
Productos	No se eliminan físicamente; se activan, inactivan o archivan.
Lógica	Reglas de mínimos, cálculos y permisos viven en servicios/modelos, no en templates.
UI/UX	Usar Component Library, tokens de color, badges y reglas responsive del Design System.
Git	Cada cambio significativo va por branch feature y PR hacia dev.
Documentación	Cambios relevantes se registran en PM y, si corresponde, en ADR, SDD o Roadmap.

Validación antes de integrar

- ☐ El módulo no duplica información de productos.
- ☐ El módulo no crea estados nuevos sin aprobación.
- ☐ La pantalla pública no muestra stock exacto.
- ☐ La UI usa botones, tablas, cards, badges y formularios consistentes.
- ☐ El cambio no introduce una tecnología distinta a las aprobadas.
- ☐ Si se tocó una regla transversal, el Project Manager fue actualizado.

16. Git Workflow

El flujo Git se mantiene simple: main es la versión estable, dev es integración, y cada sprint se trabaja en una branch feature creada desde dev. Todo cambio se revisa por Pull Request antes de integrarse.

Git Workflow aprobado

feature/<módulo> → Pull Request → dev → main

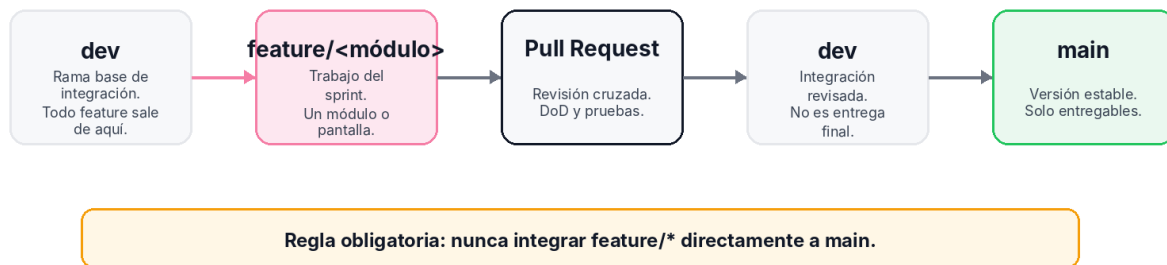


Figura 4. Git Workflow aprobado para el proyecto.

Rama	Uso	Regla
main	Versión estable y entregable.	No recibe ramas feature directamente.
dev	Integración de cambios revisados.	Recibe Pull Requests desde feature/*.
feature/<modulo>	Trabajo de un sprint, pantalla o módulo.	Debe salir de dev y volver a dev por Pull Request.

Comandos base

```

git checkout dev
git pull origin dev
git checkout -b feature/[module-name]

# trabajar y probar localmente

git status
  
```

```
git add .
git commit -m "Add [clear change description]"
git push origin feature/[module-name]
```

```
# abrir Pull Request hacia dev en GitHub
```

17. Convenciones para Branches

Las branches deben estar en inglés, en minúsculas y con guiones. El nombre debe reflejar el sprint o módulo, no el nombre de la persona que lo trabaja.

Convención	Ejemplo correcto	Evitar
feature/<nombre-corto>	feature/home	home-final
feature/<area>-<módulo>	feature/admin-products	productos-admin
feature/<flujo>-<acción>	feature/sales-payment	pago-vendedor-v2
feature/<pantalla>	feature/product-detail	detalleproducto
feature/<entrega>	feature/deploy	deploy-now

Branches oficiales sugeridas

- feature/navbar-footer
- feature/home
- feature/catalog
- feature/product-detail
- feature/quote
- feature/admin-login
- feature/admin-dashboard
- feature/admin-products
- feature/admin-inventory
- feature/admin-users
- feature/sales-search
- feature/sales-order
- feature/sales-payment
- feature/sales-receipt
- feature/deploy

Regla

Una branch debe representar trabajo revisable. Si el cambio no se puede explicar en un Pull Request pequeño, probablemente la branch es demasiado grande.

18. Convenciones para Commits

Los commits deben ser descriptivos y en inglés, tal como define el Roadmap. Se recomienda usar verbo en imperativo y explicar el resultado del cambio.

Tipo de cambio	Commit recomendado
Layout inicial	Add homepage layout

Filtros de catálogo	Implement product listing filters
Inventario	Create inventory stock update flow
Ventas	Add sales payment confirmation
Validaciones	Validate product minimum quantities
Corrección	Fix quote total calculation
UI	Improve dashboard card spacing
Documentación	Update README setup instructions

Reglas de commits

- Usar inglés claro y directo.
- Evitar mensajes genéricos como 'changes', 'fix', 'update' o 'final'.
- Hacer commits con unidades de trabajo lógicas, no todo el sprint en un solo commit gigante.
- No incluir credenciales, bases locales, archivos temporales ni configuraciones sensibles.

19. Convenciones para Pull Requests

Todo Pull Request debe abrirse desde una branch feature hacia dev. El PR debe explicar qué se hizo, cómo se probó, qué archivos se tocaron y si afecta documentación o decisiones.

Elemento	Convención
Título	Feature: Add admin product management
Destino	dev
Origen	feature/admin-products
Descripción	Resumen funcional, alcance, capturas si aplica y lista de pruebas.
Checklist	DoD, reglas de negocio, UI, pruebas y PM update.
Revisión	Revisión cruzada antes de merge.

Plantilla de Pull Request

Title: Feature: [short description]

Base: dev

Compare: feature/[module]

Summary

- ...

Scope

- ...

Files changed

- ...

Business rules validated

- ...

UI/UX validation

- ...

Tests performed

- ...

Checklist

☐ Branch comes from dev

☐ PR targets dev

☐ Commits are in English

☐ Definition of Done is complete

☐ Project Manager update is prepared

20. Buenas prácticas

Las buenas prácticas ayudan a mantener el sistema simple, explicable y profesional. No agregan reglas nuevas; facilitan implementar correctamente las reglas aprobadas.

Área	Buena práctica	Motivo
ChatGPT	Usar uno de los cuatro prompts maestros.	Evita instrucciones contradictorias.
Arquitectura	Separar routes, services, models, templates y static.	Evita archivos gigantes y mezcla de responsabilidades.
Flask	Usar Blueprints por módulo funcional.	Facilita trabajo por chats y ramas.
Templates	No colocar lógica crítica de negocio en HTML/Jinja2.	Mejora mantenimiento y reduce errores.
Datos	No duplicar productos entre módulos.	Conserva fuente única de verdad.
Inventario	Descontar stock solo al confirmar pago.	Respetar el flujo real de tienda.
UI	Aplicar Design System y Bootstrap de forma consistente.	Mantiene interfaz profesional y responsive.
Git	Abrir PR pequeño y revisable.	Reduce riesgo y acelera revisión.
Documentación	Actualizar PM al cerrar sprint.	Mantiene trazabilidad del avance.

Práctica recomendada con ChatGPT

- Pedir primero un plan breve antes de pedir código extenso.
- Solicitar cambios por archivo y en pasos pequeños.
- Pegar errores completos de consola cuando algo falle.
- Pedir revisión antes de integrar a dev.
- Cerrar cada chat con un resumen copiable al Project Manager.

21. Errores que deben evitarse

Estos errores provocan pérdida de contexto, inconsistencias entre módulos o desviaciones del alcance aprobado.

No hacer	Impacto
Abrir un chat para construir todo el sistema	Mezcla módulos, rompe trazabilidad y dificulta revisión.
Crear un prompt distinto para cada módulo	Genera redundancia y contradicciones. Solo existen cuatro

	prompts maestros.
Cambiar tecnologías aprobadas	Contradice Arquitectura Técnica y ADR.
Modificar reglas de negocio desde un chat DEV	Debe pasar por Project Manager y ADR.
Descontar inventario antes de pago	Rompe una decisión central del negocio.
Mostrar stock exacto al cliente público	Expone información sensible y contradice el proyecto.
Eliminar productos físicamente	Rompe historial de ventas y cotizaciones.
Poner cálculos críticos en templates	Dificulta pruebas y mantenimiento.
Hacer merge de feature a main	Rompe Git Workflow oficial.
Cerrar un chat sin resumen	Obliga a reconstruir contexto y retrasa el siguiente sprint.
Abrir PR sin pruebas locales	Aumenta errores de integración.
Ignorar Design System	Genera pantallas inconsistentes.

22. Checklist antes de iniciar un Sprint

Antes de abrir un chat de desarrollo, el equipo debe confirmar que el sprint está listo. Este checklist funciona como Definition of Ready.

- ☐ El Project Manager confirmó la fase y el sprint del Roadmap.
- ☐ El objetivo del sprint está definido en una frase.
- ☐ El módulo depende de componentes ya construidos o la dependencia está documentada.
- ☐ La branch feature está nombrada según la convención oficial.
- ☐ El chat fue nombrado con prefijo DEV, DOC, REV o PM según corresponda.
- ☐ Se eligió uno de los cuatro prompts maestros.
- ☐ Se revisaron Documento 0, Arquitectura Técnica, Design System, Roadmap y ADR.
- ☐ La Definition of Done del sprint es clara.
- ☐ No hay una decisión pendiente que deba resolverse antes en el PM.
- ☐ El equipo sabe qué archivos o módulos espera tocar.

Resultado esperado

Si algún punto crítico no se puede marcar, el sprint no debe empezar como desarrollo; debe resolverse primero en el Project Manager.

23. Checklist antes de cerrar un Sprint

Este checklist se usa antes de cerrar el chat DEV y actualizar el Project Manager. Funciona como Definition of Done operativo.

- ☐ El módulo cumple la funcionalidad definida en el Roadmap.
- ☐ Las rutas, servicios, modelos, templates y static están separados correctamente.
- ☐ Las reglas de negocio aprobadas fueron respetadas.
- ☐ La interfaz cumple el Design System y es responsive.
- ☐ El flujo fue probado localmente en navegador.
- ☐ No se muestran datos sensibles al público.
- ☐ No se modificó arquitectura ni tecnología sin aprobación.

- ☐ Los commits están en inglés y son descriptivos.
- ☐ El Pull Request apunta a dev.
- ☐ La revisión cruzada fue solicitada o realizada.
- ☐ El resumen de cierre está listo para el PM.
- ☐ El PM fue actualizado o queda explícitamente pendiente actualizarlo antes del próximo sprint.

24. Recomendaciones para trabajar con ChatGPT

ChatGPT debe usarse como asistente de ingeniería, no como fuente libre para cambiar decisiones. Las mejores conversaciones son específicas, acotadas y verificables.

Recomendación	Aplicación
Ser específico	Indicar módulo, sprint, branch y objetivo antes de pedir código.
Usar prompts maestros	Elegir Prompt A, B, C o D según el tipo de chat.
Pedir pasos pequeños	Primero plan, luego archivos base, luego rutas, templates, pruebas y cierre.
Pegar errores completos	Incluir traceback, comando ejecutado y archivo afectado.
Exigir coherencia	Recordar que no puede cambiar reglas de negocio ni arquitectura aprobada.
Validar localmente	No confiar solo en explicación; ejecutar Flask, revisar navegador y probar flujos.
Pedir revisión	Usar Prompt D antes de merge si el cambio toca reglas o módulos críticos.
Actualizar PM	Copiar el cierre del chat al Project Manager antes de avanzar.

Frases útiles durante un chat

- Antes de escribir código, resume el alcance y confirma que no cambiarás arquitectura ni reglas aprobadas.
- Entrega solo los archivos necesarios para este módulo y explica dónde colocarlos.
- Revisa si este cambio afecta Roadmap, ADR, SDD o Design System.
- Dame una lista de pruebas manuales para validar este flujo en Flask.
- Genera el resumen de cierre para copiar al Project Manager.

25. BIBLIOTECA OFICIAL DE PROMPTS

Esta biblioteca contiene únicamente cuatro prompts maestros para todo el proyecto. No se deben crear prompts por módulo. Productos, Inventario, Ventas, Cotizaciones, Dashboard, Login, Deploy y cualquier otro módulo usan el mismo Prompt B - Desarrollo, cambiando únicamente el nombre del módulo.

Biblioteca oficial de prompts

Todo chat futuro usa únicamente uno de estos cuatro prompts maestros



Figura 5. Biblioteca oficial de prompts maestros.

Tipo de trabajo	Prompt oficial
Generar o actualizar documentos	Prompt A - Documentación
Desarrollar cualquier módulo	Prompt B - Desarrollo
Actualizar estado general	Prompt C - Project Manager
Revisar código, arquitectura o PR	Prompt D - Revisión

Regla absoluta

Todos los futuros chats deben usar uno de estos cuatro prompts. No existe Prompt Productos, Prompt Inventario o Prompt Ventas; todos son variantes del Prompt B.

Prompt A - DOCUMENTACIÓN

Cuándo utilizarlo	Cuándo NO utilizarlo
Para crear o actualizar documentos oficiales: Arquitectura Técnica, Design System, Chat Development Guide, SDD, Roadmap, ADR u otros documentos del proyecto.	No usar para programar módulos, revisar Pull Requests, actualizar estado diario del PM o resolver errores de código.

Actúa como Software Engineering Team Lead, Scrum Master y Senior Full Stack Developer con experiencia desarrollando sistemas empresariales utilizando Flask.

Objetivo de este chat:

Construir o actualizar el documento oficial: [NOMBRE_DEL_DOCUMENTO] del proyecto "Sistema Integral de Gestión Comercial para una Importadora Mayorista".

Fuentes obligatorias:

Utiliza como contexto todos los documentos cargados en este Proyecto de ChatGPT, especialmente:

- Documento 0 - Project Context
- Documento 2 - Arquitectura Técnica
- Documento 3 - Design System, cuando aplique
- Documento 4 - Development Roadmap
- Documento 6 - Architecture Decision Record
- SDD / Documento 1, cuando esté disponible

Reglas estrictas:

- No modificar reglas de negocio aprobadas.
- No modificar arquitectura, tecnologías, estructura modular, Git Workflow ni decisiones del ADR.
- Si detectas una contradicción, detén la generación y señala qué debe revisarse en el Project Manager.
- Prioriza simplicidad, claridad, trazabilidad y formato profesional.

Entregable esperado:

- Documento completo, listo para exportarse en DOCX y PDF.
- Portada, índice, tablas, diagramas, flujos y checklists si aplican.
- Lenguaje claro para que cualquier desarrollador pueda continuar el proyecto.

Documento a construir o actualizar:

[NOMBRE_DEL_DOCUMENTO]

Alcance específico:

[OBJETIVO_DEL_DOCUMENTO]

Prompt B - DESARROLLO

Cuándo utilizarlo	Cuándo NO utilizarlo
Para construir cualquier módulo o pantalla: Configuración Flask, Productos, Inventario, Ventas, Dashboard, Cotizaciones, Login, Deploy, Catálogo, Home o Navbar.	No usar para crear documentación oficial, actualizar el estado general del proyecto o revisar PRs ya terminados.

Actúa como Senior Full Stack Developer especializado en Flask, Jinja2, Bootstrap, CSS, SQLite, SQLAlchemy, Git y sistemas empresariales modulares.

Objetivo de este chat:

Desarrollar el módulo: [NOMBRE_DEL_MÓDULO] del proyecto "Sistema Integral de Gestión Comercial para una Importadora Mayorista".

Sprint y branch:

- Sprint del Roadmap: [SPRINT]
- Branch sugerida: feature/[NOMBRE_EN_INGLÉS]

Fuentes obligatorias:

Usa todos los documentos cargados en el Proyecto de ChatGPT, especialmente Documento 0, Documento 2, Documento 3, Documento 4 y Documento 6.

Reglas que debes respetar siempre:

- No cambiar reglas de negocio, arquitectura, tecnologías, estructura modular, Git Flow ni decisiones aprobadas.
- El producto es la fuente única para catálogo, inventario, cotizaciones y ventas.
- El inventario solo se descuenta cuando un pedido está pagado.
- Los productos no se eliminan físicamente; se activan, inactivan o archivan.
- El cliente público no ve stock exacto.
- La lógica de negocio va en services/models; los templates solo presentan datos.
- La UI debe respetar el Design System con Bootstrap y CSS propio.
- El trabajo debe ser modular, entendible y fácil de revisar por Pull Request.

Forma de trabajo:

1. Resume el alcance del módulo y confirma dependencias.
2. Propón un plan corto antes de escribir código.
3. Indica archivos a crear o modificar.
4. Entrega código por pasos pequeños.
5. Incluye instrucciones para probar localmente.
6. Sugiere commits en inglés.
7. Prepara el resumen de cierre para el Project Manager.

Entrega final del chat:

- Archivos creados/modificados.
- Código necesario.
- Pruebas manuales realizadas o sugeridas.
- Mensajes de commit en inglés.
- Descripción de Pull Request hacia dev.
- Resumen de cierre para actualizar el Project Manager.

Módulo a desarrollar:

[NOMBRE_DEL_MÓDULO]

Prompt C - PROJECT MANAGER

Cuándo utilizarlo	Cuándo NO utilizarlo
Para actualizar el estado general del proyecto, Roadmap, ADR, SDD, avance por sprint, decisiones, riesgos, bloqueos y próximo sprint.	No usar para escribir código de un módulo ni para hacer revisión técnica detallada de un PR.

Actúa como Project Manager, Scrum Master y Software Engineering Team Lead del proyecto "Sistema Integral de Gestión Comercial para una Importadora Mayorista".

Objetivo de este chat:

Actualizar el estado general del proyecto usando el resumen del chat cerrado y mantener coherencia entre Roadmap, ADR, SDD, documentación y próximo sprint.

Fuentes obligatorias:

Usa todos los documentos cargados en el Proyecto de ChatGPT, especialmente:

- Documento 0 - Project Context
- Documento 2 - Arquitectura Técnica
- Documento 3 - Design System
- Documento 4 - Development Roadmap
- Documento 6 - Architecture Decision Record
- SDD / Documento 1, cuando esté disponible

Entrada de actualización:

[PEGAR AQUÍ EL RESUMEN DE CIERRE DEL CHAT DEV, DOC O REV]

Debes entregar:

1. Estado actual del proyecto.
2. Fase y sprint actualizados del Roadmap.
3. Estado del módulo trabajado: completo, en revisión, bloqueado o pendiente.
4. Cambios que deben reflejarse en el Roadmap.
5. Decisiones nuevas o cambios que deben registrarse en el ADR.
6. Cambios que impactan SDD, Arquitectura Técnica o Design System.
7. Riesgos, bloqueos o deuda técnica.
8. Próximo sprint recomendado.
9. Nombre del próximo chat a abrir.
10. Prompt maestro que debe usarse en el próximo chat.

Reglas estrictas:

- No inventar funcionalidades fuera del alcance aprobado.
- No aprobar cambios de arquitectura sin registrar decisión.
- Si falta información, pedir una actualización puntual al chat que cerró el sprint.
- Mantener el proceso simple: Roadmap -> Chat -> Desarrollo -> Pruebas -> Commit -> PR -> PM -> Siguiente Sprint.

Prompt D - REVISIÓN

Cuándo utilizarlo	Cuándo NO utilizarlo
Para revisar código, arquitectura, Pull Requests, errores, mejoras, consistencia UI/UX, reglas de negocio, seguridad y Git Workflow.	No usar para iniciar un módulo nuevo, generar documentos oficiales completos ni actualizar estado general del PM.

Actúa como Senior Full Stack Reviewer, Software Architect y Tech Lead especializado en Flask, Jinja2, Bootstrap, CSS, SQLite, SQLAlchemy, Git y sistemas empresariales modulares.

Objetivo de este chat:

Revisar el código, arquitectura o Pull Request de: [NOMBRE_DEL_MÓDULO_O_PR] del proyecto "Sistema Integral de Gestión Comercial para una Importadora Mayorista".

Fuentes obligatorias:

Usa todos los documentos cargados en el Proyecto de ChatGPT, especialmente Documento 0, Documento 2, Documento 3, Documento 4 y Documento 6.

Revisa obligatoriamente:

- Cumplimiento del Roadmap y Definition of Done.
- Arquitectura modular: routes, services, models, templates y static separados.
- Uso correcto de Flask Blueprints, Jinja2, Bootstrap, CSS propio y SQLite.
- Reglas de negocio aprobadas: producto único, inventario al pagar, no eliminación física, stock público no exacto.
- Seguridad por rol y protección de información sensible.
- Consistencia visual con el Design System.
- Validaciones, manejo de errores y casos borde.
- Commits en inglés, branch feature y Pull Request hacia dev.
- Impacto en ADR, SDD, Roadmap o documentación.

Formato de salida:

1. Veredicto: Aprobado / Cambios requeridos / Bloqueado.
2. Resumen de revisión.
3. Hallazgos bloqueantes.
4. Hallazgos importantes.
5. Mejoras sugeridas.
6. Archivos o zonas afectadas.
7. Checklist de reglas aprobadas.
8. Cambios solicitados con criterio de aceptación.
9. Recomendación final para merge o devolución.

Reglas estrictas:

- No aprobar un PR que haga merge directo a main.
- No aprobar un cambio que modifique reglas de negocio sin ADR.
- No aprobar una pantalla pública que muestre stock exacto.
- No aprobar un flujo que descuente inventario antes del pago.
- No proponer tecnologías fuera de las aprobadas.

Material a revisar:

[PEGAR CÓDIGO, DIFF, DESCRIPCIÓN DEL PR O ERROR]

26. Cierre operativo

Este documento convierte el trabajo con ChatGPT en un proceso repetible, simple y controlado. La guía existe para que cualquier integrante pueda abrir un chat nuevo, escoger el prompt correcto, trabajar un módulo, revisar resultados y actualizar el Project Manager sin perder contexto.

Resumen final

El proyecto se desarrolla con una sola metodología: un Project Manager, chats separados por objetivo, Git Flow feature → PR → dev, y cuatro prompts maestros: Documentación, Desarrollo, Project Manager y Revisión.

Regla final de continuidad

Antes de iniciar cualquier conversación futura, el usuario debe identificar qué tipo de trabajo realizará. Si es documento, usa Prompt A. Si es módulo, usa Prompt B. Si es estado general, usa Prompt C. Si es revisión, usa Prompt D. Esta decisión simple mantiene todo el proyecto coherente hasta el despliegue y entrega final.